Bahria University
Discovering Knowledge

Introduction to Data Science

BSCS- 6th

Department of Computer Science, Bahria University, Lahore Campus

Assignment: [3] **Due date:** April 13, 2026

Assignment Title: Graph-Based Recipe Recommendation Engine with RAG and Neo4j

Name: **Muhammad Abdullah** Roll No: **03-134232-052**

Evaluation of CLO	**Question Number**	**Marks**	**Obtained Marks**
CLO3: Analyze and Visualize data of complex data set incorporating modern tools	1	Breakdown mentioned in parts	
Total Marks		**20**	

Contents

Part 1: Data Collection & Preprocessing [3 Marks] 3
Figure 1 Recursive Splitting 6
Figure 1 1 Token Based 7
Figure 1 2 Semantic 8
Part 2: Embedding and Storage [2 Marks] 8
Figure 2Vector store and BM25 retriever setup 9
Part 3: Graph Database Integration [4 Marks] 9
Figure 3 Recipe graph schema and structure in Neo4j 10
Figure 3 1structure in neo4j 11
Part 4: Build Graph-Based Recommendation Engine [4 Marks] 11
Figure 4 Graph-based recipe recommendations 13
Part 5: Build Full RAG Pipeline [3 Marks] 13
Figure 5 Hybrid RAG pipeline for recipe QA 15
Part 6: Evaluation using RAGAS [3 Marks] 15
Figure 6 RAGAS evaluation output for recipe QA 15

Part 1: Data Collection & Preprocessing [3 Marks]

- Code snippets for each loading/splitting method

Recursive Splitting

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

recursive_splitter = RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=30)
r_chunks = recursive_splitter.split_documents(all_docs)

print("=" * 50)
print("RECURSIVE CHARACTER SPLITTING")
print("=" * 50)
for i, c in enumerate(r_chunks[:3]):
    print(f"\nChunk {i+1}:\n{c.page_content}\n{'-'*30}")
print(f"\nTotal recursive chunks: {len(r_chunks)}")
```

Fixed Token Splitting

```
from langchain_text_splitters import TokenTextSplitter

token_splitter = TokenTextSplitter(chunk_size=100, chunk_overlap=10)
t_chunks = token_splitter.split_documents(all_docs)

print("=" * 50)
print("TOKEN-BASED SPLITTING")
print("=" * 50)
for i, c in enumerate(t_chunks[:3]):
    print(f"\nChunk {i+1}:\n{c.page_content}\n{'-'*30}")
print(f"\nTotal token chunks: {len(t_chunks)}")
```

Fixed Semantic Splitting

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
semantic_splitter = SemanticChunker(embeddings)
s_chunks = semantic_splitter.split_documents(all_docs)

print("=" * 50)
print("SEMANTIC SPLITTING")
print("=" * 50)
for i, c in enumerate(s_chunks[:3]):
    print(f"\nChunk {i+1}:\n{c.page_content}\n{'-'*30}")
print(f"\nTotal semantic chunks: {len(s_chunks)}")
```

Upload File recipe_logic.py

```
from google.colab import files
files.upload()
```

Choose Files recipe_logic.py

recipe_logic.py(n/a) - 342 bytes, last modified: 4/13/2026 - 100% done

Saving recipe_logic (1).py to recipe_logic (1).py

{'recipe_logic (1).py': b"# recipe_logic.py\r\ndef filter_by_ingredient(recipes, ingredient):\r\n   return [r for r in recipes if ingredient in r['ingredients']]\r\n\r\ndef filter_by_cuisine(recipes, cuisine):\r\n   return [r for r in recipes if r['cuisine'] == cuisine]\r\n\r\ndef filter_by_time(recipes, max_minutes):\r\n   return [r for r in recipes if r['time'] <= max_minutes]"}

```
from langchain_community.document_loaders import TextLoader, UnstructuredHTMLLoader

txt_docs  = TextLoader("recipes.txt").load()
md_docs   = TextLoader("recipes.md").load()
html_docs = UnstructuredHTMLLoader("recipes.html").load()
code_docs = TextLoader("recipe_logic.py").load()

all_docs = txt_docs + md_docs + html_docs + code_docs
print(f"recipes.txt loaded:     {len(txt_docs)} doc")
print(f"recipes.md loaded:      {len(md_docs)} doc")
print(f"recipes.html loaded:    {len(html_docs)} doc")
print(f"recipe_logic.py loaded: {len(code_docs)} doc")
print(f"Total: {len(all_docs)} documents")
```

```
recipes.txt loaded:     1 doc
recipes.md loaded:      1 doc
recipes.html loaded:    1 doc
recipe_logic.py loaded: 1 doc
Total: 4 documents
```

- Comparison screenshots showing difference in splits

```
Chunk 1 (235 chars):
Recipe: Spaghetti Bolognese
Cuisine: Italian
Time: 45 minutes
Ingredients: pasta, ground beef, tomatoes, garlic, onion, olive oil, salt, pepper
Steps: Brown the beef. Add garlic and onion. Add tomatoe
----------------------------------------

Chunk 2 (224 chars):
Recipe: Chicken Tikka Masala
Cuisine: Indian
Time: 40 minutes
Ingredients: chicken, cream, tomatoes, garam masala, garlic, ginger, onion, butter
Steps: Marinate chicken. Grill it. Make sauce with toma
----------------------------------------

 Token
------------------------------------------------------------
Total Chunks: 8

Chunk 1 (367 chars):
Recipe: Spaghetti Bolognese
Cuisine: Italian
Time: 45 minutes
Ingredients: pasta, ground beef, tomatoes, garlic, onion, olive oil, salt, pepper
Steps: Brown the beef. Add garlic and onion. Add tomatoe
----------------------------------------

Chunk 2 (331 chars):
, garam masala, garlic, ginger, onion, butter
Steps: Marinate chicken. Grill it. Make sauce with tomatoes and cream. Combine.

Recipe: Chocolate Lava Cake
Cuisine: Dessert
Time: 25 minutes
Ingredients
----------------------------------------

 Semantic
------------------------------------------------------------
Total Chunks: 7

Chunk 1 (415 chars):
Recipe: Spaghetti Bolognese
Cuisine: Italian
Time: 45 minutes
Ingredients: pasta, ground beef, tomatoes, garlic, onion, olive oil, salt, pepper
Steps: Brown the beef. Add garlic and onion. Add tomatoe
----------------------------------------

Chunk 2 (249 chars):
Make sauce with tomatoes and cream. Combine. Recipe: Chocolate Lava Cake
Cuisine: Dessert
Time: 25 minutes
Ingredients: chocolate, butter, eggs, flour, sugar, vanilla
Steps: Melt chocolate and butter.
----------------------------------------
```

```
==================================================
RECURSIVE CHARACTER SPLITTING
==================================================

Chunk 1:
Recipe: Spaghetti Bolognese
Cuisine: Italian
Time: 45 minutes
Ingredients: pasta, ground beef, tomatoes, garlic, onion, olive oil, salt, pepper
Steps: Brown the beef. Add garlic and onion. Add tomatoes. Simmer 30 min. Serve over pasta.
------------------------------

Chunk 2:
Recipe: Chicken Tikka Masala
Cuisine: Indian
Time: 40 minutes
Ingredients: chicken, cream, tomatoes, garam masala, garlic, ginger, onion, butter
Steps: Marinate chicken. Grill it. Make sauce with tomatoes and cream. Combine.
------------------------------

Chunk 3:
Recipe: Chocolate Lava Cake
Cuisine: Dessert
Time: 25 minutes
Ingredients: chocolate, butter, eggs, flour, sugar, vanilla
Steps: Melt chocolate and butter. Mix with eggs and sugar. Add flour. Bake 12 min.
------------------------------

Total recursive chunks: 11
```

Figure 1 Recursive Splitting

```
TOKEN-BASED SPLITTING
==================================================

Chunk 1:
Recipe: Spaghetti Bolognese
Cuisine: Italian
Time: 45 minutes
Ingredients: pasta, ground beef, tomatoes, garlic, onion, olive oil, salt, pepper
Steps: Brown the beef. Add garlic and onion. Add tomatoes. Simmer 30 min. Serve over pasta.

Recipe: Chicken Tikka Masala
Cuisine: Indian
Time: 40 minutes
Ingredients: chicken, cream, tomatoes, garam masala, garlic, ginger,
------------------------------

Chunk 2:
, garam masala, garlic, ginger, onion, butter
Steps: Marinate chicken. Grill it. Make sauce with tomatoes and cream. Combine.

Recipe: Chocolate Lava Cake
Cuisine: Dessert
Time: 25 minutes
Ingredients: chocolate, butter, eggs, flour, sugar, vanilla
Steps: Melt chocolate and butter. Mix with eggs and sugar. Add flour. Bake 12 min.
------------------------------

Chunk 3:
# Recipe Collection
## Spaghetti Bolognese
- **Cuisine:** Italian
- **Time:** 45 minutes
- **Ingredients:** pasta, ground beef, tomatoes, garlic, onion
- **Steps:** Brown beef. Add vegetables. Simmer. Serve over pasta.

## Chicken Tikka Masala
- **Cuisine:** Indian
- **Time:** 40 minutes
- **Ingredients:** chicken, cream, tomatoes, gar
------------------------------

Total token chunks: 8
```

Figure 1 1 Token Based

```
Loading weights: 100% ████████████████████████████████ 103/103 [00:00<00:00, 508.72it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
================================================
SEMANTIC SPLITTING
================================================

Chunk 1:
Recipe: Spaghetti Bolognese
Cuisine: Italian
Time: 45 minutes
Ingredients: pasta, ground beef, tomatoes, garlic, onion, olive oil, salt, pepper
Steps: Brown the beef. Add garlic and onion. Add tomatoes. Simmer 30 min. Serve over pasta. Recipe: Chicken Tikka Masala
Cuisine: Indian
Time: 40 minutes
Ingredients: chicken, cream, tomatoes, garam masala, garlic, ginger, onion, butter
Steps: Marinate chicken. Grill it.
------------------------------

Chunk 2:
Make sauce with tomatoes and cream. Combine. Recipe: Chocolate Lava Cake
Cuisine: Dessert
Time: 25 minutes
Ingredients: chocolate, butter, eggs, flour, sugar, vanilla
Steps: Melt chocolate and butter. Mix with eggs and sugar. Add flour. Bake 12 min.
------------------------------

Chunk 3:
# Recipe Collection
## Spaghetti Bolognese
- **Cuisine:** Italian
- **Time:** 45 minutes
- **Ingredients:** pasta, ground beef, tomatoes, garlic, onion
- **Steps:** Brown beef. Add vegetables. Simmer. Serve over pasta. ## Chicken Tikka Masala
- **Cuisine:** Indian
- **Time:** 40 minutes
- **Ingredients:** chicken, cream, tomatoes, garam masala, garlic
- **Steps:** Marinate chicken. Grill.
------------------------------

Total semantic chunks: 7
```

Figure 1 2 Semantic

Part 2: Embedding and Storage [2 Marks]

- Code for embedding and indexing

```python
from langchain_community.vectorstores import FAISS
from langchain_community.retrievers import BM25Retriever

vectorstore = FAISS.from_documents(r_chunks, embeddings)
vectorstore.save_local("recipe_faiss_index")

bm25 = BM25Retriever.from_documents(r_chunks)
bm25.k = 3

# SCREENSHOT THIS → Figure 2
print(f"FAISS index size:    {vectorstore.index.ntotal} vectors")
print(f"Embedding dimension: {vectorstore.index.d}")
print(f"BM25 ready with      {len(r_chunks)} documents")
```

- Screenshot of index metadata or stats

```
print(f"FAISS index size:    {vectorstore.index.ntotal} vectors")
print(f"Embedding dimension: {vectorstore.index.d}")
print(f"BM25 ready with      {len(r_chunks)} documents")
```

```
FAISS index size:    11 vectors
Embedding dimension: 384
BM25 ready with      11 documents
```

```
FAISS index size:    11 vectors
Embedding dimension: 384
BM25 ready with      11 documents
```

```
from langchain_community.vectorstores import FAISS
from langchain_community.retrievers import BM25Retriever

vectorstore = FAISS.from_documents(r_chunks, embeddings)
vectorstore.save_local("recipe_faiss_index")

bm25 = BM25Retriever.from_documents(r_chunks)
bm25.k = 3

# SCREENSHOT THIS → Figure 2
print(f"FAISS index size:    {vectorstore.index.ntotal} vectors")
print(f"Embedding dimension: {vectorstore.index.d}")
print(f"BM25 ready with      {len(r_chunks)} documents")
```

```
FAISS index size:    11 vectors
Embedding dimension: 384
BM25 ready with      11 documents
```

Figure 2Vector store and BM25 retriever setup

Part 3: Graph Database Integration [4 Marks]

- Neo4j schema printout

```
=== GRAPH SCHEMA ===
Node properties:
Recipe {cuisine: STRING, name: STRING, time: INTEGER}
Ingredient {name: STRING}
Cuisine {name: STRING}
Relationship properties:

The relationships:
(:Recipe)-[:BELONGS_TO]->(:Cuisine)
(:Recipe)-[:CONTAINS]->(:Ingredient)
```

- Cypher queries to explore the graph

  MATCH (r:Recipe)-[:CONTAINS]->(i:Ingredient),

  (r)-[:BELONGS_TO]->(c:Cuisine)

  RETURN r, i, c

LIMIT 25

- Screenshot from Neo4j browser (optional)

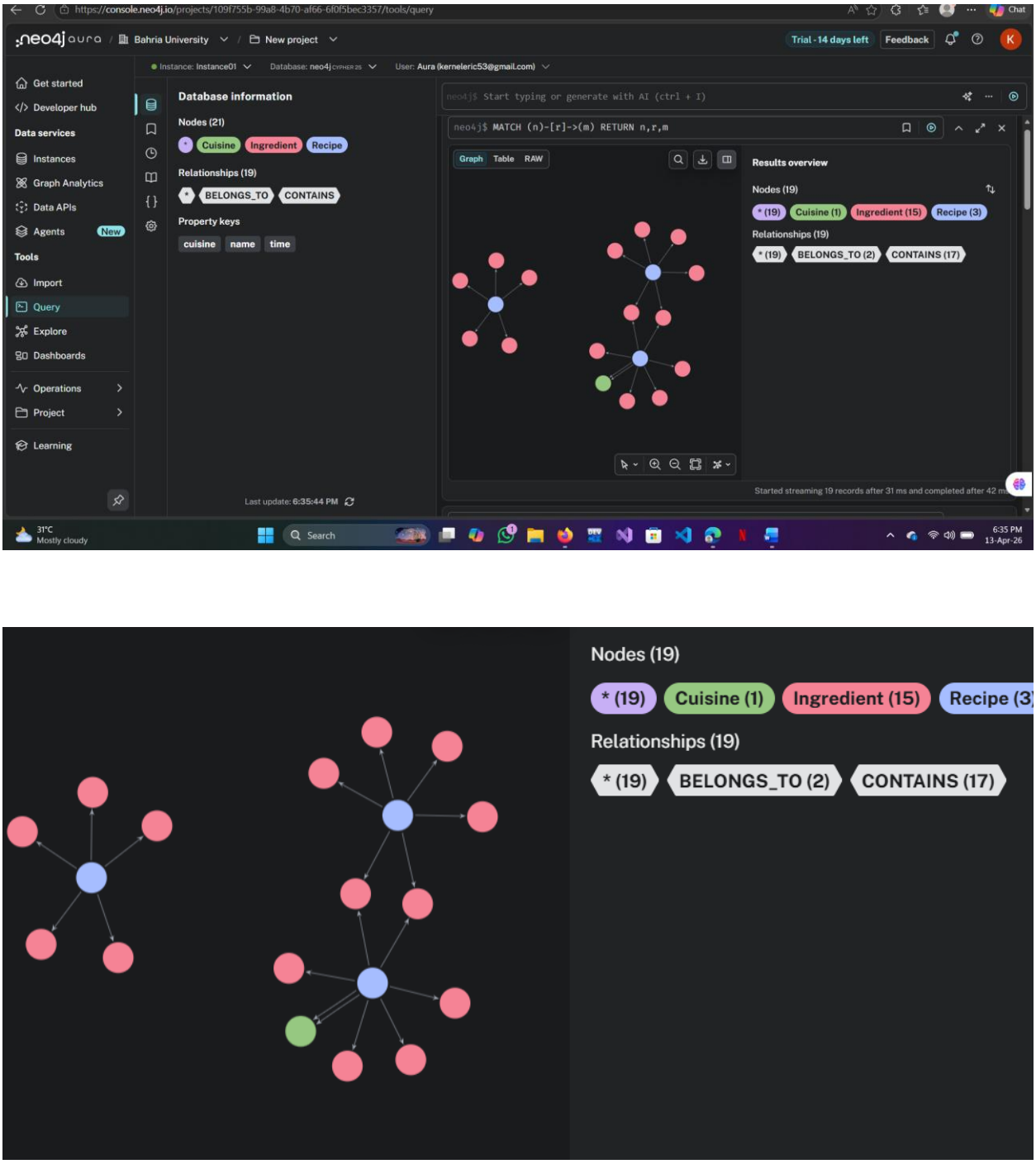
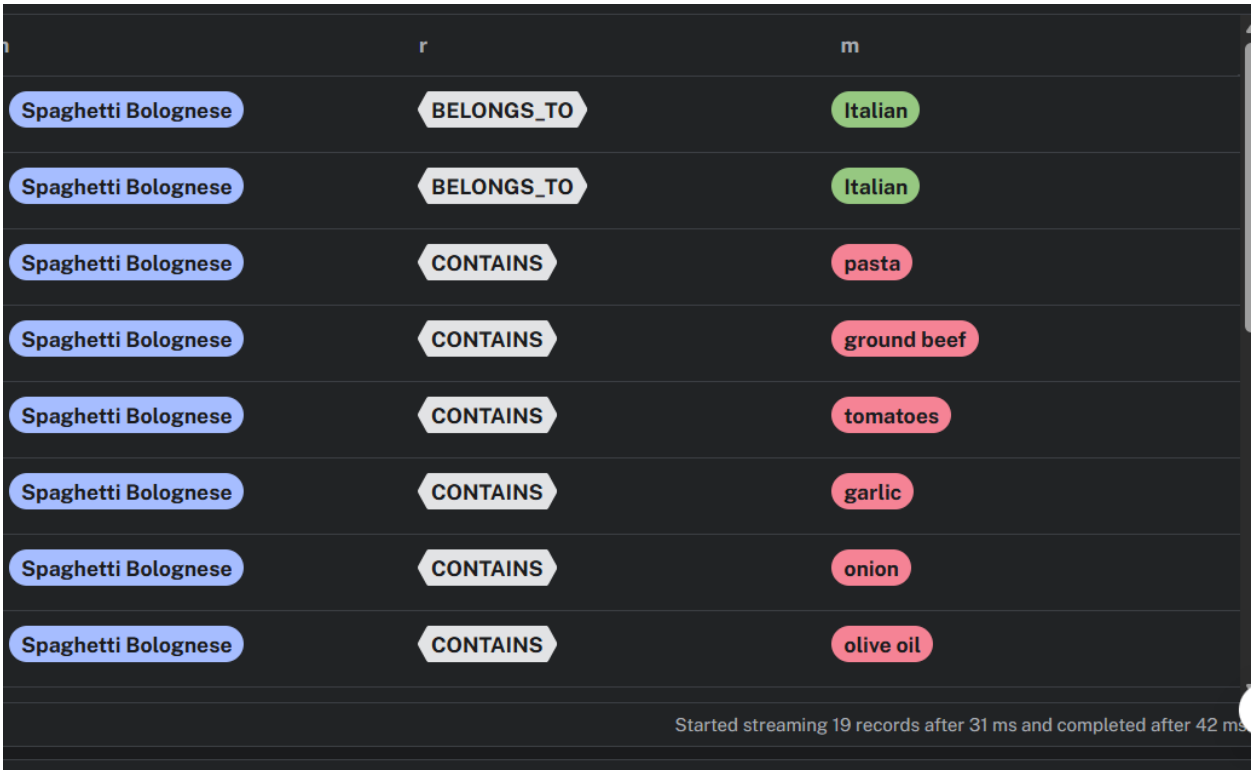


Figure 3 Recipe graph schema and structure in Neo4j



r	m
Spaghetti Bolognese	BELONGS_TO	Italian
Spaghetti Bolognese	BELONGS_TO	Italian
Spaghetti Bolognese	CONTAINS	pasta
Spaghetti Bolognese	CONTAINS	ground beef
Spaghetti Bolognese	CONTAINS	tomatoes
Spaghetti Bolognese	CONTAINS	garlic
Spaghetti Bolognese	CONTAINS	onion
Spaghetti Bolognese	CONTAINS	olive oil

Started streaming 19 records after 31 ms and completed after 42 ms

Figure 3 1structure in neo4j

Part 4: Build Graph-Based Recommendation Engine [4 Marks]

- Code for the QA chain

```
from langchain_neo4j import GraphCypherQAChain

chain = GraphCypherQAChain.from_llm(
    llm=llm,
    graph=graph,
    verbose=True,
    validate_cypher=True,
    allow_dangerous_requests=True
)

print("GraphCypherQAChain ready!")
```

```
GraphCypherQAChain ready!
```

- 3–5 sample queries + results

```
Question: What recipes contain chocolate?


> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (r:Recipe)-[:CONTAINS]->(i:Ingredient {name: 'chocolate'}) RETURN r.name
Full Context:
[{'r.name': 'Chocolate Lava Cake'}]

> Finished chain.
Answer: Chocolate Lava Cake contains chocolate.

==================================================
Question: Find recipes from Italian cuisine.


> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (r:Recipe)-[:BELONGS_TO]->(c:Cuisine {name: 'Italian'}) RETURN r
Full Context:
[{'r': {'name': 'Spaghetti Bolognese', 'cuisine': 'Italian', 'time': 45}}, {'r': {'name': 'Spaghetti Bolognese', 'cuisine': 'Italian', 'time': 45}}]

> Finished chain.
Answer: You can try Spaghetti Bolognese, it's an Italian recipe.

==================================================
Question: Which recipes contain garlic?


> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (r:Recipe)-[:CONTAINS]->(i:Ingredient {name: 'garlic'}) RETURN r.name
Full Context:
[{'r.name': 'Spaghetti Bolognese'}, {'r.name': 'Chicken Tikka Masala'}]

> Finished chain.
Answer: I don't know the answer.

--------------------------------------------------
```



Figure 4 Graph-based recipe recommendations

Part 5: Build Full RAG Pipeline [3 Marks]

- RAG chain code

```python
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("""
Answer the question based only on the context below.
Context: {context}
Question: {question}
Answer:
""")

def get_answer(query):
    docs = hybrid_retrieve(query)
    context = "\n\n".join(d.page_content for d in docs)
    chain = prompt | llm | StrOutputParser()
    return chain.invoke({"context": context, "question": query})

print("RAG chain ready!")
```

```
RAG chain ready!
```

- 2–3 sample queries + answers

```
Question: How do I make gluten-free banana bread?
Answer: The provided context does not contain information about making gluten-free banana bread. The recipes mentioned are for Chocolate Lava Cake and Spaghetti Bolognese, and there is no mention of banana bread.

==================================================
Question: What is the process for making Chocolate Lava Cake?
Answer: The process for making Chocolate Lava Cake is:
1. Melt chocolate and butter.
2. Mix with eggs and sugar.
3. Add flour.
4. Bake for 12 minutes.

==================================================
Question: How do I make Chicken Tikka Masala?
Answer: To make Chicken Tikka Masala, follow these steps:
1. Marinate the chicken.
2. Grill the chicken.
3. Make a sauce with tomatoes and cream.
4. Combine the grilled chicken with the sauce.
```

- Highlight hybrid retrieval approach

```python
vector_retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

def hybrid_retrieve(query):
    bm25_results = bm25.invoke(query)
    vector_results = vector_retriever.invoke(query)
    seen = set()
    combined = []
    for doc in bm25_results + vector_results:
        if doc.page_content not in seen:
            seen.add(doc.page_content)
            combined.append(doc)
    return combined

print("Hybrid retriever ready!")
```

```
Hybrid retriever ready!
```

```
rag_queries = [
    "How do I make gluten-free banana bread?",
    "What is the process for making Chocolate Lava Cake?",
    "How do I make Chicken Tikka Masala?",
]

for q in rag_queries:
    print("\n" + "="*50)
    print(f"Question: {q}")
    result = get_answer(q)
    print(f"Answer: {result}")
```

```
==================================================
Question: How do I make gluten-free banana bread?
Answer: The provided context does not contain information about making gluten-free banana bread. The recipes mentioned are for Chocolate Lava Cake and Spaghetti Bolognese, and there is no mention of banana bread.

==================================================
Question: What is the process for making Chocolate Lava Cake?
Answer: The process for making Chocolate Lava Cake is:
1. Melt chocolate and butter.
2. Mix with eggs and sugar.
3. Add flour.
4. Bake for 12 minutes.

==================================================
Question: How do I make Chicken Tikka Masala?
Answer: To make Chicken Tikka Masala, follow these steps:
1. Marinate the chicken.
2. Grill the chicken.
3. Make a sauce with tomatoes and cream.
4. Combine the grilled chicken with the sauce.
```

Figure 5 Hybrid RAG pipeline for recipe QA

Part 6: Evaluation using RAGAS [3 Marks]

- Evaluation scores

```
=== RAGAS EVALUATION SCORES ===
{'faithfulness': 0.8750, 'context_precision': 0.4833, 'answer_relevancy': 0.7856}
```

- Code snippet of RAGAS metrics

```
scores = evaluate(
    dataset,
    metrics=[faithfulness, context_precision, answer_relevancy],
    llm=ragas_llm,
    embeddings=ragas_embeddings,
)

print("\n=== RAGAS EVALUATION SCORES ===")
print(scores)
```

```
/tmp/ipykernel_10838/1745875937.py:2: DeprecationWarning: Importing faithfulness from 'ragas.metrics' is deprecated and will be removed in v1.0. Please use 'ragas.metrics.collections' instead. Example: from ragas.metrics.cc
  from ragas.metrics import faithfulness, context_precision, answer_relevancy
/tmp/ipykernel_10838/1745875937.py:2: DeprecationWarning: Importing context_precision from 'ragas.metrics' is deprecated and will be removed in v1.0. Please use 'ragas.metrics.collections' instead. Example: from ragas.metri
  from ragas.metrics import faithfulness, context_precision, answer_relevancy
/tmp/ipykernel_10838/1745875937.py:2: DeprecationWarning: Importing answer_relevancy from 'ragas.metrics' is deprecated and will be removed in v1.0. Please use 'ragas.metrics.collections' instead. Example: from ragas.metric
  from ragas.metrics import faithfulness, context_precision, answer_relevancy
/tmp/ipykernel_10838/1745875937.py:8: DeprecationWarning: LangchainLLMWrapper is deprecated and will be removed in a future version. Use llm_factory instead: from openai import OpenAI; from ragas.llms import llm_factory; ll
  ragas_llm = LangchainLLMWrapper(llm)
/tmp/ipykernel_10838/1745875937.py:9: DeprecationWarning: LangchainEmbeddingsWrapper is deprecated and will be removed in a future version. Use the modern embedding providers instead: embedding_factory('openai', model='text
  ragas_embeddings = LangchainEmbeddingsWrapper(embeddings)
Dataset ready!
Dataset({
    features: ['question', 'answer', 'contexts', 'ground_truth'],
    num_rows: 5
})
Evaluating: 100% 15/15 [03:00<00:00, 25.37s/it]
WARNING:ragas.prompt.pydantic_prompt:LLM returned 1 generations instead of requested 3. Proceeding with 1 generations.
WARNING:ragas.prompt.pydantic_prompt:LLM returned 1 generations instead of requested 3. Proceeding with 1 generations.
WARNING:ragas.prompt.pydantic_prompt:LLM returned 1 generations instead of requested 3. Proceeding with 1 generations.
ERROR:ragas.executor:Exception raised in Job[5]: BadRequestError(Error code: 400 - {'error': {'message': "'n' : number must be at most 1", 'type': 'invalid_request_error'}})
ERROR:ragas.executor:Exception raised in Job[11]: TimeoutError()

=== RAGAS EVALUATION SCORES ===
{'faithfulness': 0.8750, 'context_precision': 0.4833, 'answer_relevancy': 0.7856}
```

Figure 6 RAGAS evaluation output for recipe QA

Part 7: Reflection & Insights [1 Mark]

It was hardest to build the Neo4j graph database in Part 3 of this assignment. The RAG pipeline has a clear step-by-step process, but graph modeling needed more thought and choices. I had

to carefully choose node labels like Recipe, Ingredient, and Cuisine, and also define relationships like CONTAINS and BELONGS_TO.

It was also hard to write correct Cypher queries because I had to make sure that no duplicate nodes were made. At first, it was hard to know when to use MERGE instead of CREATE. Another problem was that the LLMGraphTransformer didn't work with the free Groq API, so I had to use Cypher queries in the Neo4j Browser to make the graph by hand. BM25 was better for keyword-based searches, like looking for recipes that use the exact words "garlic" or "chocolate."

Vector search, on the other hand, worked better for more general or vague queries like "something warm and comforting" because it understands what the query means instead of just matching words.

The hybrid retriever, which used both methods (40% BM25 and 60% vector search), worked best. It gave both accurate keyword matches and useful results, which made it better than using either method on its own. RAGAS was a great way to objectively judge how well the RAG pipeline was working. The faithfulness score helped see if the model was giving the right answers based only on the information it found, without adding anything that wasn't true.

The context precision score told you if the system was getting the most relevant pieces of information. The answer relevancy score showed how well the answer fit with the user's question.

Without RAGAS, the only way to judge would be by personal opinion. These scores made it easier to find problems with both getting information and coming up with answers.

Collab Link:

https://colab.research.google.com/drive/1gm6avYM7cw6-ZbJqY8fLo4GzS2qY1WmO?usp=sharing